C++ Programming

Juha Yli-Hemminki

27.11.2025

SeAMK

Containers

- Container is an object that is used for store collection of other objects (container elements)
- Containers are typically implemented as template classes – type of the elements can be defined
- Container manages its size
- Containers have different kinds of member functions to manage elements
- Examples: list, vector, map

SeAMK

List

- Linked list (double linked) implementation
- Each element has link to previous and next element in the list
- Elements may be stored in arbitrary locations in the memory
- Extra memory usage since each element needs value, next pointer, prev pointer
- No random access (list[i])

List functions

- push_back – add element at the end
- pop_back – delete last element
- push_front – add element at begining
- pop_front – delete first element
- unique – remove duplicates
- sort – sort the list

- clear – delete all elements

SeAMK

Vector

- An array that can change its size
- Dynamically allocated array – reallocation may be needed when size is changed (may have some extra capacity)
- Elements have continuous locations in the memory like in basic array
- Good in adding/removing at the end, worse when adding/removing in other locations
- Fast random access (vector[i])

SeAMK

Vector functions

- push_back – add element at the end
- pop_back – delete last element
- sort – sort elements
- clear – delete all elements
- operator [] – to access element at certain position like in array

SeAMK

Map (and unordered_map)

- Map is another type of container like vector
- Each element in map has a **key** value and mapped **value**
- Key values in map are unique, no duplicates
- Types for keyvalue and mapped value are given when declaring map (map<int, int> …)

Map

- Some functions to use with map m:
  - m.size() returns number of elements in the map
  - m.erase(key) removes element by key
  - m.clear() removes all element from map
  - m.find(key) finds the element for the key, returns iterator

SeAMK

Map

- Add new items into the map:
  - m.insert(pair<int, int>(key, value) to make pair of key and value and add item (in this case type for key and value both int)
  - m.insert({“key”, 55}) does the same
  - Or then:
  - m[“key”] = 55; // [] operator

Map

- Add new items into the map:

Goal	Use
Simple assignment / overwriting	`operator[]`
Insert only if missing	`insert()`
Insert efficiently without creating temporary	`emplace()`
Insert only if missing + avoid constructing value unnecessarily	`try_emplace()`
Insert or update	`insert_or_assign()`
Old-style pair construction	`make_pair()`

Map

- Old way to loop map:

```
for (std::map<std::string, int>::iterator it = m.begin(); it != m.end(); ++it) {
    std::cout << it->first << " = " << it->second << "\n";
}
```

- Modern way to loop map:

```
for (const auto& [key, value] : m) {
    std::cout << key << " = " << value << "\n";
}
```

Map

- To handle result for find() iterator is needed:

```
std::map<int, int>::iterator it = m.find(key);
```

- Modern way:

```
auto it = m.find(key);
```

- Alternative to find is contains() to inspect if key exists:

```
if(m.contains(key)) {}
```

SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES